CHAPTER 03 · 학습 노트

성능을 좌우하는 DB 설계와 쿼리

『주니어 백엔드 개발자가 반드시 알아야 할 실무 지식』 3장 (51~88쪽 전체) 개념 정리

이 장에서 다루는 내용

1. 성능에 핵심인 DB
2. 조회 트래픽을 고려한 인덱스 설계 (단일·복합, 선택도, 커버링)
3. 조회 성능 개선 방법 (미리 집계, ID 기준 조회, 시간 범위 제한, count 생략, 오래된 데이터 정리, 장비 확장, 캐시 서버)
4. 알아두면 좋을 주의 사항 (타임아웃, 복제 DB, 배치, 타입 조인, 테이블 변경, 최대 연결 수)
5. 실패와 트랜잭션 고려하기
6. 복습 퀴즈

※ 원문을 옮겨 적은 것이 아니라, 책의 내용을 학습용으로 재구성·요약한 노트입니다. 그림은 개념을 설명하기 위해 새로 그렸습니다.

1. 성능에 핵심인 DB

사례 · H 서비스의 출석 체크 대란

교사·학생 대상 서비스. 코로나로 재택 수업이 시작되며 가입자가 급증했고, 개학과 함께 출석 체크 시간(8시 40분)이 다가오면 응답이 10초를 넘겨 에러가 쏟아졌다.

원인: 사용자가 조금만 늘어도 DB CPU가 90%를 넘었다. 호출 빈도가 높은 기능에 **풀 스캔**을 일으키는 쿼리가 있었고, 데이터가 쌓이며 쿼리가 느려지고 트래픽 증가로 DB 부하까지 커져 응답 시간이 계속 늘어났다.

알아두기 · 풀 스캔(full scan)

테이블의 모든 데이터를 순서대로 읽는 것. 주로 where 조건에 맞는 **인덱스가 없을 때** 발생하며, 인덱스보다 전체를 읽는 편이 빠르다고 DB가 판단해도 발생한다. 데이터가 적을 땐 티가 안 나지만, 데이터가 늘면 어느 순간 응답 시간이 **기하급수적으로** 늘어난다 → 설계할 때 항상 풀 스캔 가능성을 염두에 둔다.

DB 성능은 연결된 **모든 서버**의 성능에 영향을 준다. 쿼리가 느려져 전체 서비스가 느려지는 일은 흔하지만, DB 자체가 문제인 경우보다 **DB를 잘못 사용한 경우**가 훨씬 많다. 서버 개발자가 조금만 신경 써도 DB 성능 문제를 크게 줄이거나 없앨 수 있다.

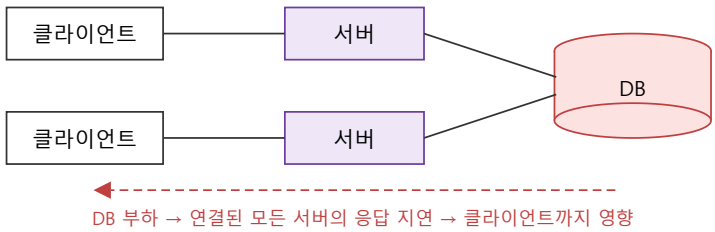


그림 1 (책 그림 3.1). DB 성능 문제는 전체 서비스로 번진다

2. 조회 트래픽을 고려한 인덱스 설계

대부분 시스템은 **조회 비율이 높다**(게시판: 다수는 읽기만, 일부만 쓰기·수정·삭제). 같은 게시판이라도 인기 커뮤니티와 사내 공지 게시판은 사용자 수·글 수가 전혀 다르다. 테이블을 설계할 때는 **조회 기능과 트래픽 규모**를 함께 고려해야 한다.

article (책 그림 3.2)	
id	integer(10)
category	integer(10)
writerId	integer(10)
title	varchar(255)
content	clob
regdt	timestamp

카테고리별 목록 조회 쿼리:

```
select id, category, writerId, title, content
from article
where category = 10
order by id desc limit 20, 10
```

category에 인덱스가 없다면?

	사내 공지 게시판	인기 커뮤니티
데이터량	주 1건 → 1년 52건, 10년 해도 **520건**	하루 5,000건 × 약 6년 ≈ **1,000만 건**
트래픽	직원 1,000명이 5분 안에 읽는다고 해도 약 **3.34 TPS**	다수 사용자가 동시에 목록 조회
결론	풀 스캔해도 문제없음 → **인덱스 불필요**	매번 1,000만 건 비교, 풀 스캔이 겹쳐 DB CPU 100% → **category 인덱스 필수**

원칙: **조회 패턴을 기준으로** 인덱스를 설계한다. '내가 쓴 글 목록'도 마찬가지로 writerId를 포함한 인덱스가 필요하다.

```
select id, category, writerId, title, content from article
where writerId = 10 order by id desc limit 10, 10
```

알아두기 · 전문 검색 인덱스

제목에 단어가 들어간 글을 찾는 `where title like '%검색어%'` 처럼 **중간 포함 like**는 풀 스캔을 일으킨다. 엘라스틱서치 같은 검색 엔진을 쓰면 DB 없이 검색을 구현할 수 있고, 별도 검색 엔진이 어렵다면 DB의 전문(full-text) 검색 기능(Oracle Text, MySQL FULLTEXT 인덱스)으로 풀 스캔 없이 문자열 검색을 할 수 있다.

2-1. 단일 인덱스와 복합 인덱스

activityLog (책 그림 3.3)	
id (PK)	integer(10)
userId	integer(10)
activityType	varchar(20)
activityDate	date
activityDatetime	timestamp
memo	varchar(200)

일평균 방문 10만 명 × 평균 5회 활동 → 하루 **50만 건**, 한 달 **1,500만 건**. 대규모는 아니지만, 알맞은 인덱스가 없으면 문제가 생기는 규모다.

예 1) 고객센터: 특정 사용자의 일자별 활동 조회

```
select * from activityLog
where userId = 123 and activityDate = '2024-07-31'
order by activityDatetime desc
```

- userId를 포함한 인덱스는 필수. 고민은 activityDate까지 넣을지 여부 → **단일 (userId)** vs **복합 (userId, activityDate)**
- **사용자당 데이터량**을 가늠해 판단한다
  - 주 1회 방문·5건 → 1년 260건, 5년 1,500건 → 수천 건 이하 비교는 0.1초 이내 → **단일 인덱스로 충분**
  - 매일 방문·30건 이상 → 1년 1만 건 이상, 사용자당 수만 건 → **복합 인덱스** 필요

예 2) 통계: 특정 일자의 활동 타입별 개수

```
select activityDate, activityType, count(activityType)
from activityLog
where activityDate = '2024-07-28'
group by activityType;
```

- 풀 스캔을 피하려면 activityDate 인덱스가 필요. activityType 포함 여부는 **실행 빈도와 실행 시간**으로 결정한다.
- activityDate만 인덱스면 그날 데이터 50만 건을 모두 읽어 10초 이상 걸릴 수 있다. 하지만 **하루 한 번** 실행해 결과를 별도 테이블에 저장한다면 30초가 걸려도 괜찮다.
- 하루 데이터가 수백만 건으로 커지면 **(activityDate, activityType) 복합 인덱스**를 적용 → 인덱스 값만으로 결과를 구할 수 있다(커버링 인덱스, 실제 데이터를 읽지 않아 빠름).

2-2. 선택도를 고려한 인덱스 칼럼 선택

선택도(selectivity) = 인덱스 칼럼의 고유한 값 비율. 높을수록 고유 값이 많고, 인덱스 조회 효율이 높다.

```
create table member (
  memberId int not null primary key,
  gender char(1),
  ...
);
select * from member where gender = 'F' and birthyear = 1900;
```

gender 값이 M 50만, F 50만, N 1천 개라면 gender 인덱스로도 여전히 **50만 건**을 확인해야 한다 → 선택도가 낮아 효율이 떨어진다.

예외 · 선택도가 낮아도 인덱스가 맞는 경우 — 작업 큐

```
create table jobqueue (
  jobid varchar(16) not null primary key,
  status char(1) not null,  -- W(대기) / P(처리 중) / C(완료)
  ...
);
select * from jobqueue where status = 'W' order by jobid asc;
```

고유 값이 3개뿐이라 선택도는 낮다. 그러나 대부분은 C이고 W·P는 소수이며, 작업 실행기는 **W만 반복 조회**한다. 인덱스가 없으면 매번 풀 스캔 → 모든 작업이 지연된다. 따라서 선택도와 상관없이 status를 인덱스로 추가해야 한다.

2-3. 커버링 인덱스 활용하기

커버링 인덱스: 특정 쿼리 실행에 필요한 칼럼을 **모두 포함**하는 인덱스. 실제 데이터를 읽지 않고 인덱스만으로 결과를 만들 수 있어 쿼리 효율이 높아진다.

```
select * from activityLog
where activityDate = '2024-07-31' and activityType = 'VISIT';
```

(activityDate, activityType) 인덱스가 있으면 읽을 데이터는 **빠르게 선택**하지만, select * 이라 칼럼값을 얻으려고 결국 각 행의 **실제 데이터를 읽어 와야** 한다.

```
select activityDate, activityType
from activityLog
where activityDate = '2024-07-31' and activityType = 'VISIT';
```

이 쿼리는 필요한 칼럼이 **모두 인덱스에 있어서** 실제 데이터에 접근하지 않는다 → 데이터 읽는 과정이 생략돼 실행이 빨라진다. 이것이 커버링 인덱스다.

2-4. 인덱스는 필요한 만큼만 만들기

자주 쓰는 쿼리가 다음 두 개라고 하자.

```
select * from activityLog
where userId = 123 and activityDate = '2024-07-31'
order by activityDatetime desc;

select * from activityLog
where userId = 123 and activityDate = '2024-07-31' and activityType = 'BUY'
order by activityDatetime desc;
```

둘 다 빠르게 하려고 `(userId, activityDate)`와 `(userId, activityDate, activityType)` 두 복합 인덱스를 만들 수 있다. 그러나 두 번째 인덱스가 효과를 보려면 **한 사용자가 하루에 만드는 데이터가 성능에 영향을 줄 만큼 많아야** 한다. 하루 수백 개 미만이면 효과를 체감하기 어렵다.

⚠ **효과가 적은 인덱스는 오히려 손해**
인덱스는 조회를 빠르게 하지만 데이터 **추가·변경·삭제 때마다 인덱스 관리 비용(시간)**이 붙는다. 인덱스도 데이터라서 많아질수록 **메모리·디스크 사용량**도 늘어난다.

요구사항을 살짝 바꿔 기존 인덱스 활용하기

새 쿼리가 기존 인덱스를 쓰지 못한다면, 인덱스를 추가하기 전에 **요구사항을 일부 변경**할 수 있는지 검토한다.

```
create table reservation (
  id bigint not null primary key,
  name varchar(30) not null,
  reserveDate date not null,
  state char(2) not null,
  regDt timestamp not null
);
create index idxRdate ON reservation (reserveDate);

-- 예약자 이름으로 조회: name 인덱스가 없어 풀 스캔
select * from reservation where name = '예약자 이름' order by regDt desc

-- 요구사항 변경: "특정 일자에 예약한 예약자 이름으로 조회"
select * from reservation
where reserveDate = '2024-08-09' and name = '고객명';
```

바뀐 쿼리는 reserveDate가 들어간 **idxRdate 인덱스**를 쓴다. 그 날짜의 데이터만 비교하므로 새 인덱스 없이도 풀 스캔을 피할 수 있다.

COLUMN · 같은 칼럼에 대한 인덱스 또 추가하지 않기
오래 쓴 테이블엔 기능을 붙이면서 인덱스가 계속 늘고, 담당 개발자도 바뀐다. 어떤 인덱스가 왜 있는지 지식이 전해지지 않아 사라지기도 한다. 그러다 보면 **같은 칼럼 인덱스를 중복**으로 만들거나 거의 같은 효과의 인덱스를 또 만든다(저자도 두 인덱스가 같은 칼럼을 쓰는 테이블을 본 적이 있다).
같은 칼럼 인덱스가 2개라고 조회가 두 배 빨라지지 않고 **관리 비용만** 는다. 새 인덱스를 만들기 전에 기존 인덱스와 같은 효과를 낼 인덱스가 있는지 먼저 검토하자.

3. 조회 성능 개선 방법

인덱스가 아니어도 조회 성능을 개선할 방법이 있다. 어렵지 않고 쉽게 쓸 수 있는 방법들을 살펴본다.

3-1. 미리 집계하기

간단한 설문 조사 기능 요구사항: ① 설문마다 질문 4개 고정 ② 회원은 설문마다 '좋아요' 가능 ③ 설문 목록에 답변 수와 '좋아요' 수 표시.

테이블 (책 그림 3.4)	칼럼
survey	surveyId (PK), subject, from, to, question1~4
answer	surveyId + memberId (PK), answer1~4 varchar(100)
liked	surveyId + memberId (PK)

목록에 답변 수·좋아요 수를 보여 주려고 select 절에 **서브 쿼리**를 쓰면:

```
select s.id, s.subject,
  (select count(*) from answer a where a.surveyId = s.id) as answerCnt,
  (select count(*) from liked l where l.surveyId = s.id) as likeCnt
from survey s
order by id desc
limit 30;
```

논리적으로는 맞지만 성능이 문제다. 설문 30개, 설문당 평균 답변자 10만 명, 좋아요 1만 명이라면:

- 목록 조회 1번
- 답변자 수 세는 서브 쿼리 30번 (각각 10만 개를 셈)
- 좋아요 수 세는 서브 쿼리 30번 (각각 1만 개를 셈)

총 61번 실행 · 쿼리 시간 = 0.01 + 0.1 × 30 + 0.05 × 30 = 4.51초

사용자가 많은 서비스에서 4.51초는 매우 길고, 트래픽이 몰리면 급격히 더 느려진다. 해결은 간단하다: count·sum 같은 집계를 조회 시점에 하지 말고 **미리 계산해 별도 칼럼에 저장**한다. survey 테이블에 답변자 수·좋아요 수 칼럼을 추가하고, 이벤트가 생길 때마다 값을 바꾼다.

survey + 집계 칼럼 (책 그림 3.5)	
surveyId (PK)	integer(10)
subject	varchar(100)
from / to	date
question1~4	varchar(255)
answerCnt	integer(10)
likedCnt	integer(10)

```
-- 답변 추가 + 카운트 증가
insert into answer values (...);
update survey set answerCnt = answerCnt + 1
 where surveyId = 아이디;

-- 좋아요 취소 + 카운트 감소
delete from liked where answerId = 설문ID
 and memberId = 회원ID;
update survey set likedCnt = likedCnt - 1
 where surveyId = 설문ID;
```

```
select s.id, s.subject, answerCnt, likeCnt
from survey s order by id desc limit 30;
```

목록 쿼리에 서브 쿼리가 필요 없어지므로, 그만큼 실행 시간이 줄어 조회가 빨라진다.

COLUMN · 비정규화해도 괜찮나요?

정규화는 중복을 줄이고 무결성을 높이기 위해 데이터를 구조화하는 것. answerCnt는 answer 테이블로 구할 수 있는 값을 **중복** 저장한 것이고, 실제 개수와 어긋날(무결성이 깨질) 수도 있다.
그래도 설문 예에서는 실시간 집계 칼럼이 맞다고 본다. 실제 10,150개인데 10,149로 보여도 목록을 보는 회원에게는 심각하지 않다. 정확한 값이 필요한 최종 보고서는 관리툴에서 answer·liked 테이블로 실제 개수를 구하면 된다.

COLUMN · 동시성 문제는 없나요?

`update survey set answerCnt = answerCnt + 1 where surveyId = 1;` 을 5개 클라이언트가 동시에 실행하면? DB가 **원자적(atomic)**으로 처리하면 정확히 5 증가하지만, 그렇지 않으면 결과를 예측할 수 없다. 트랜잭션 격리 수준과 DBMS에 따라 다르므로, 증감 쿼리를 쓸 때는 **사용 중인 DBMS·격리 수준에서 원자적으로 처리되는지 검증**해야 한다.

3-2. 페이지 기준 대신 ID 기준 목록 조회

게시글 10만 개를 10개씩 보여 주면 1만 페이지. 첫 페이지(`limit 10 offset 0`)는 10개만 읽지만, 마지막 페이지는:

```
select id, subject, writer, regdt from article
order by id desc
limit 10 offset 99990;
```

DB는 어떤 id가 99,991번째인지 모르므로 **99,990개를 센 뒤** 10개를 읽는다 → 세는 시간만큼 느려진다.

인덱스에 없는 칼럼이 where에 있으면 더 느리다. 예) `where deleted = false ... limit 10 offset 10000`

- deleted가 인덱스에 없으면 각 행을 읽어 값을 확인해야 하고, true인 행도 섞여 있어 10,000개를 세려면 10,000개 넘게 읽는다.
- deleted를 인덱스에 넣어도 **오프셋만큼 세는 시간**은 그대로 남는다.

해결: 특정 ID를 기준으로 조회

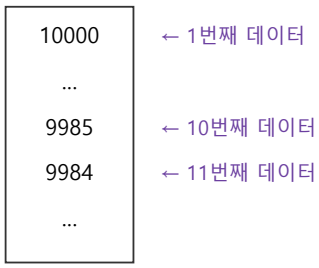


그림 2 (책 그림 3.6). deleted=false인 데이터의 id 분포 예

```
-- 첫 10개 (마지막 id = 9985)
select * from article
where deleted = false
order by id desc limit 10;

-- 다음 10개: 마지막 id 기준
select * from article
where id < 9985 and deleted = false
order by id desc limit 10;
```

id는 PK 인덱스라 9985보다 작은 9984를 **바로 찾는다**. 오프셋을 세는 과정이 생략되어 빠르다. 스크롤로 다음 데이터를 불러오는 **모바일 화면**과 특히 잘 맞는다(페이지 번호 대신 마지막 ID를 받아 조회).

> **TIP · "다음 데이터 있음" 여부 내려주기**
> 10개를 보여 줄 때 **11개를 조회**한다(`where id < 1001 and deleted = false order by id desc limit 11`). 결과가 11개면 앞 10개만 응답하고 다음 존재 여부 = true, 10개 이하면 false.

3-3. 조회 범위를 시간 기준으로 제한하기

포털 뉴스처럼 **일자별**로 목록을 제공하면 조회 범위가 하루로 제한된다.

```
select title, ... from news
where regdt >= '2024-08-09 00:00:00' and regdt < '2024-08-10 00:00:00'
order by regdt desc limit 100;
```

- regdt 인덱스만 있으면 된다. 하루 기사 수가 많지 않다면 `and category = '01'` 같은 조건을 더해도 인덱스 추가 없이 성능이 유지된다.
- 쇼핑몰 주문 내역도 30개씩 순서대로 대신 **월별** 조회 → 인덱스 (customerId, orderDt)로 쿼리가 단순하고 빠르다.

```
select id, ... from order
where customerId = 100
  and orderDt >= '2024-08-01 00:00:00' and orderDt < '2024-09-01 00:00:00'
order by orderDt desc;
```

최신 데이터만 조회하기

대부분 기능은 최신 데이터를 주로 본다. 매달 제공되는 점검 결과도 고객은 최근 결과에만 관심이 있으므로 최근 6개월만 제공해도 된다.

```
select * from checkResult
where cust_id = ? and checkMonth >= ?
order by checkMonth desc;
```

공지사항도 3년 전 것은 거의 안 읽는다. 최신 데이터만 보여 줄 수 있다면 **제품 담당자와 협의**해 기능을 줄이고 구현을 단순화하자. (예: 구글 '최근 보안 활동'은 최근 28일만 보여 준다 — 책 그림 3.7)

> **덤 효과 · DB 메모리 캐시 효율 향상**
> DB는 조회한 데이터를 메모리에 캐시한다. 최신 데이터가 많이 조회되므로 캐시에 올라 있을 확률이 높고, 범위를 최신으로 제한하면 캐시 효율과 응답 속도가 함께 좋아진다.

3-4. 전체 개수 세지 않기

```
select id, ... from member where name like '지은%' order by id desc limit 20;
select count(*) from member where name like '지은%';
```

데이터가 적을 땐 괜찮지만, 데이터가 늘수록 count 시간도 늘어난다. 조건에 해당하는 데이터를 **모두 탐색**해야 하기 때문이다(커버링 인덱스여도 인덱스 전체 스캔, 아니면 실제 데이터를 전부 읽음). 조회가 느려지는 원인

이 전체 개수 쿼리라면 실행하지 않는 방법을 고려하고, 서비스 담당자와 **전체 개수를 표시하지 않는 방향**으로 협의한다.

> **COLUMN · 평소 패턴을 숙지하고 이상 징후 찾기**
>
> 저자는 APM(Application Performance Monitoring)을 늘 띄워 둔다. 평소 응답 시간 약 0.5초, 히트맵의 응답 시간 **띠**도 0.5초에 있었는데, 어느 날 띠가 살짝 위(약 0.7초)로 올라갔다. 당장 문제 될 속도는 아니라 그냥 넘길 수도 있었던 변화였다.
> 원인은 한 쿼리가 0.2초 늘어난 것 — 서브 쿼리의 count가 데이터 증가로 느려졌다. **미리 집계** 방식으로 바꾸자 응답이 0.4초로 줄었다. 평소 패턴에 관심을 두면 문제 징후를 일찍 잡을 수 있다.

3-5. 오래된 데이터 삭제 및 분리 보관

데이터가 늘면 쿼리 시간도 는다. 반대로 **개수를 일정하게 유지**하면 실행 시간도 일정하게 유지된다. 과거 데이터 삭제가 그 방법이다.

- 예) **로그인 시도 내역**: 낯선 지역 로그인 알림 등 이상 징후 탐지용이라 최근 몇 달 치면 충분. 필요한 게 180일이면 매일 181일 이전 데이터를 삭제 → 일관된 서비스 품질.
- 내부 관리 시스템에서 이전 데이터가 필요하면, 서비스 DB엔 180일만 두고 그 이전은 **별도 저장소로 분리 보관**한다.

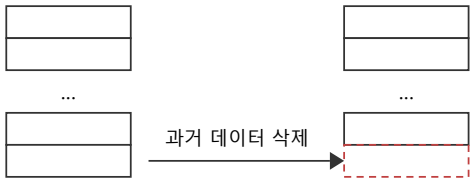


그림 3 (책 그림 3.8). 과거 데이터를 지워 데이터 개수를 일정 수준으로 유지

> **알아두기 · 단편화와 최적화**
>
> DELETE를 해도 일반적으로 DB의 디스크 사용량은 줄지 않는다(삭제 표시만 하고 공간은 나중에 재사용). 추가·변경·삭제가 반복되면 데이터가 흩어지고 빈 공간이 생기는 **단편화(fragmentation)**가 생겨 디스크 I/O 증가로 성능이 떨어지고, 실제 데이터보다 더 많은 공간을 쓰게 된다. **최적화** 작업으로 데이터를 재배치하면 단편화가 줄고 물리적 디스크 사용량도 줄어든다.

3-6. DB 장비 확장하기

개선 방법을 못 찾았거나 시간이 필요하면, 일단 **수직 확장**으로 서비스를 살려 두고 개선할 시간을 번다. 클라우드면 빠른 CPU·더 많은 코어·메모리 증설을 짧은 시간에 할 수 있고 효과도 눈에 띈다. 비용은 늘지만 서비스를 못 하는 것보다 낫다.

> **COLUMN · 장비발**
>
> 저자의 신입 시절, 오라클 DB에서 평소 3~5초 걸리던 쿼리가 간헐적으로 5초를 넘기 시작했다. 예정된 DB 장비 교체 후 같은 쿼리가 **0.5초 미만**으로 줄었다.

수평 확장도 가능하다. 조회 비중이 높은 서비스라면 **주 DB-복제 DB(Primary-Replica)** 구조로 처리량을 효과적으로 늘린다.

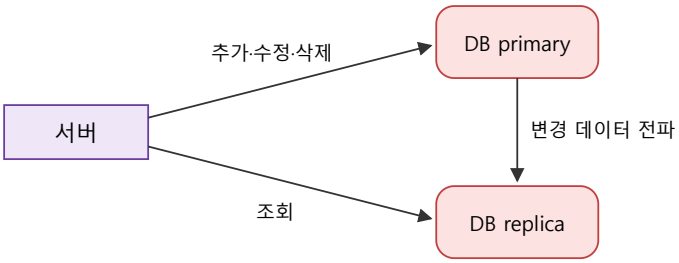


그림 4 (책 그림 3.9). 읽기 전용 복제 DB를 추가해 조회 처리량을 늘린다

서버는 변경 쿼리는 주 DB, 조회 쿼리는 복제 DB로 실행한다. 조회 트래픽이 늘면 복제 DB를 추가한다.

MEMO

DB 서버는 보통 API 서버보다 몇 배 좋은(비싼) 장비다. 일시적 조회 급증 때문에 수평 확장하면 **고정 비용**이 계속 부담되므로, 비용 대비 이점이 확실할 때 확장한다.

COLUMN · 우리 회사에는 DBA가 없어요

복제 DB 구성은 DB 운영에 익숙지 않은 서버 개발자에게 부담스러운 작업이다(저자도 DBA가 처리). DBA 채용이 어려운 작은 회사라면 **DB 전문 회사와 컨설팅 계약**을 맺어 정기 점검·장애 대응·운영 지원을 받는 것도 방법이다.

3-7. 별도 캐시 서버 구성하기

트래픽이 급증하면 여러 개선책을 써도 DB만으로는 한계가 오고, 비용 때문에 DB를 더 키우지 못할 수도 있다. 이럴 땐 **캐시 서버**를 고려한다.

- 대규모 트래픽 서비스 대부분이 캐시 서버를 기본으로 사용한다.
- 대규모가 아니어도 캐시를 잘 쓰면 DB 확장보다 **적은 비용**으로 더 많은 트래픽을 처리할 수 있고, 레디스 같은 캐시 서버 구성이 DB 확장보다 부담도 적다.
- 코드 수정이 필요하지만, 얻는 처리량이 크다면 그쪽이 합리적인 선택이다.

4. 알아두면 좋을 몇 가지 주의 사항

4-1. 쿼리 타임아웃

응답 시간이 길어지면 처리량만 떨어지는 게 아니다. 쿼리가 15초 이상 걸리면 사용자는 몇 초 만에 느리다고 느끼고 **재시도**한다. 앞 요청이 아직 처리 중인데 새 요청이 들어오므로 동시 요청 수가 **기하급수적으로** 늘고 서버 부하가 폭증한다.

- 대책: **쿼리 실행 시간 제한(타임아웃)**. 예) 5초로 제한하면 초과 시 에러 → 사용자는 에러 화면을 보지만, 서버 입장에선 요청이 종료된 것이라 재시도해도 동시 요청이 폭증하지 않는다.
- 기능 특성에 따라 다르게: 블로그 글 조회는 몇 초 이내로 짧게, **결제**는 더 길게 — 결제 중 타임아웃이 나면 후속 처리와 데이터 정합성이 복잡해진다.

4-2. 상태 변경 기능은 복제 DB에서 조회하지 않기

'변경은 주 DB, 조회는 복제 DB'를 잘못 이해해 **모든 SELECT를 복제 DB**로 보내면 두 가지 문제가 생긴다.

① **복제 지연**: 주 DB의 변경은 (네트워크로 복제 DB에 전달 → 복제 DB가 반영) 두 단계를 거치므로 시간이 걸리고, 그동안 두 DB의 값이 다르다.

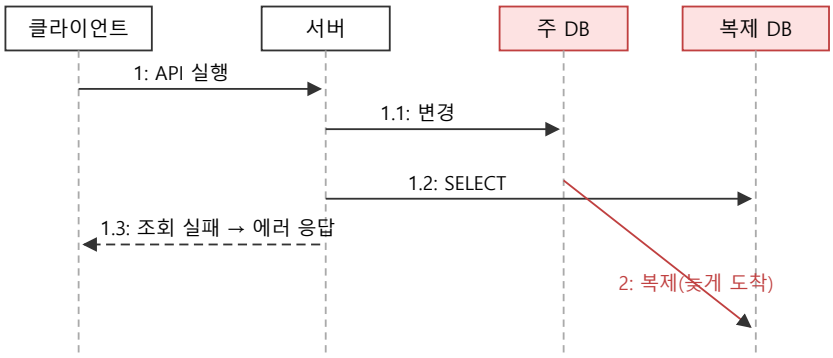


그림 5 (책 그림 3.10). 변경 직후 복제 DB에서 읽으면 아직 반영 전일 수 있다

② **트랜잭션 문제**: 복제는 트랜잭션 **커밋 시점**에 이뤄진다. 주 DB 트랜잭션 안에서 데이터를 바꾸고, 변경 대상이 될 데이터를 복제 DB에서 조회하면 불일치로 문제가 생긴다.

> **원칙**
> 회원 가입·변경·등록·삭제처럼 INSERT/UPDATE/DELETE를 하는 기능에서 변경 대상 데이터를 조회해야 한다면, **변경 쿼리를 실행하는 주 DB에서 SELECT**한다.

4-3. 배치 쿼리 실행 시간 증가

배치는 일괄 조회·집계·생성(일별 회원 통계, 사용량 기반 요금 계산 등)을 하며 group by + count/sum, 조건부 일괄 변경 쿼리를 자주 쓴다. 데이터가 늘면 30초 → 몇 분 → 몇 십 분으로 늘고, 집계 쿼리는 메모리를 많이 써서 **임계점을 넘으면** 몇 시간이 지나도 안 끝날 수 있다.

- 예방: 배치 쿼리의 **실행 시간을 지속적으로 추적**해 급증을 감지한다.
- 가장 빠른 해결은 DB 사양 업그레이드지만 항상 가능하진 않다. 대안 2가지:
  - **커버링 인덱스 활용**: 집계 대상 칼럼이 인덱스에 있으면 인덱스만 스캔 → 빠르고 DB 메모리도 덜 씀

- **데이터를 일정 크기로 나눠 처리**

```
-- 한 달 치를 한 번에 → 임계점을 넘으면 끝나지 않을 수 있음
select ... 각종 집계 from accessLog al
where al.accessDatetime >= '2024-08-01 00:00:00'
  and al.accessDatetime < '2024-09-01 00:00:00'
group by ...

-- 하루 단위로 나눠 1일~말일 실행 후 합치기 (필요하면 1시간·10분 단위)
select ... 각종 집계 from accessLog al
where al.accessDatetime >= '2024-08-01 00:00:00'
  and al.accessDatetime < '2024-08-02 00:00:00'
group by ...
```

나눠 처리하면 짧은 간격으로 집계할 수도 있다. 예) 새벽 일괄 처리 대신 **10분 간격 증분 집계**:

1. 통계 테이블에 반영된 마지막 accessDatetime을 구한다
2. [마지막 accessDatetime, 마지막 + 10분] 구간의 accessLog를 집계한다
3. 2의 결과를 통계 테이블에 반영한다

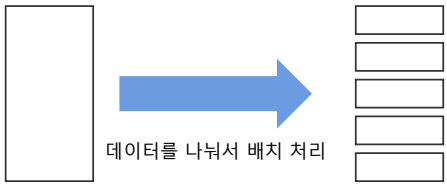


그림 6 (책 그림 3.11). DB 확장이 어렵다면 나눠서 처리

COLUMN · 오늘 매출 집계 데이터가 없어요!
I사의 자체 매출 집계 시스템. 회계 부서가 오늘 매출 데이터가 없다고 연락했는데, 담당자는 해외여행 중. 로그를 보니 한 쿼리가 3시간 넘게 실행 중이었다. DB 확장은 불가.
쿼리는 group by·집계 함수·서브 쿼리가 복잡했고, `insert … select`로 조회와 변경을 동시에 해서 **테이블 잠금 시간**까지 길었다.
조치: ① insert…select를 select와 insert로 분리 → select만 먼저 실행해 정상 완료 확인 ② 결과를 한 트랜잭션이 아니라 **일정 개수씩 나눠** 반영. → 3시간 넘던 작업이 **약 30분**으로 단축. 이어진 쿼리들도 범위를 작게 나눠 매출 집계를 마칠 수 있었다.

4-4. 타입이 다른 칼럼 조인 주의

user	
userId (PK)	integer(10)
name	integer(10)

push (책 그림 3.12)	
id (PK)	integer(10)
receiverType	varchar(2)
receiverId (인덱스)	varchar(200)

push는 푸시 발송 내역. 수신 타입마다 식별자 타입이 달라 receiverId를 varchar로 정의했고, 자주 조회하므로 인덱스를 걸었다.

```
select u.userId, u.name, p.*
from user u, push p
where u.userId = 145
  and u.userId = p.receiverId
  and p.receiverType = 'MEMBER'
order by p.id desc limit 100;
```

인덱스를 쓸 것 같지만 **integer(userId) vs varchar(receiverId)**라 DB가 비교할 때마다 각 행의 receiverId를 integer로 **타입 변환**한다 → 인덱스를 온전히 활용하지 못하고, 인덱스만 스캔하더라도 데이터가 많으면 변환 때문에 오래 걸린다. 해결: 비교 칼럼의 타입을 맞춘다(MySQL 예).

```
  and cast(u.userId as char character set utf8mb4)
      collate 'utf8mb4_unicode_ci' = p.receiverId
```

MEMO

문자열끼리 비교할 때도 **캐릭터셋이 같은지** 확인한다. 캐릭터셋이 다르면 그 자체로 변환이 일어날 수 있다.

4-5. 테이블 변경은 신중하게

데이터가 많은 테이블에 칼럼을 추가하거나 열거 타입 칼럼을 바꿀 때는 **정말로** 주의해야 한다. 무심코 바꿨다가 서비스가 장시간 멈출 수 있다.

- 이유: 예컨대 MySQL은 테이블을 바꿀 때 새 테이블을 만들고 원본 데이터를 복사한 뒤 교체한다. 복사 중에는 UPDATE/INSERT/DELETE 같은 **DML이 허용되지 않아** 그 시간만큼 서비스가 멈춘다.
- DML을 허용하며 변경하는 기능(온라인 DDL)도 있지만 항상 가능하진 않다 → 데이터가 많으면 **점검 시간**을 잡고 변경하는 게 가장 안정적이다.
- DBA가 있으면 점검 필요 여부를 판단해 주지만, 없다면 스스로 판단해야 하니 더 신중해야 한다.

COLUMN · 서비스를 멈추게 한 테이블 변경

수천만 건 테이블의 열거 타입 칼럼을 변경하면서 온라인 DDL을 쓰지 않아 복사가 시작됐다. 수십 분이 지나도 안 끝났고, 여러 기능이 쓰는 테이블이라 서비스가 정상 동작하지 않아 점검 모드로 전환. 몇 시간 뒤 주 DB 변경이 끝났지만 **복제 DB 변경**은 여전히 진행 중이라 추가 시간이 필요했다. 단 한 줄의 변경 쿼리가 **8시간 가까이** 서비스를 멈추게 했고, 데이터 보정·고객 보상 같은 후처리가 며칠간 이어졌다.

4-6. DB 최대 연결 개수

상황: API 서버 3대, 트래픽 증가로 수평 확장 필요, DB CPU는 20%로 여유. 그런데 새로 추가한 API 서버가 DB 커넥션 생성에 실패한다면? → DB의 **최대 연결 개수**를 확인한다.

DB 최대 연결 100 · 서버당 풀 30 × 4대 = 120 필요 → 20개 연결 실패

이 경우 DB의 최대 연결 개수만 늘려도 해결된다. **단**, DB CPU 사용률이 **70% 이상**이면 연결 수를 늘리면 안 된다(연결이 많을수록 부하 증가 → 성능 저하). 먼저 캐시 서버 구성이나 쿼리 튜닝으로 DB 부하를 낮춘 뒤 필요할 때 늘린다.

5. 실패와 트랜잭션 고려하기

초보가 자주 놓치는 것이 **DB 트랜잭션**이다. 코드가 늘 정상 동작하는 건 아니므로 비정상 상황의 트랜잭션 처리를 반드시 고민해야 한다. 흔한 실수는 **트랜잭션 없이 여러 데이터를 수정**하는 것 — 실무에서도 자주 보이고, 실수로 빠뜨리기도 한다. DB 코드를 쓸 때는 트랜잭션의 시작과 종료 경계를 명확히 설정했는지 반드시 확인한다.

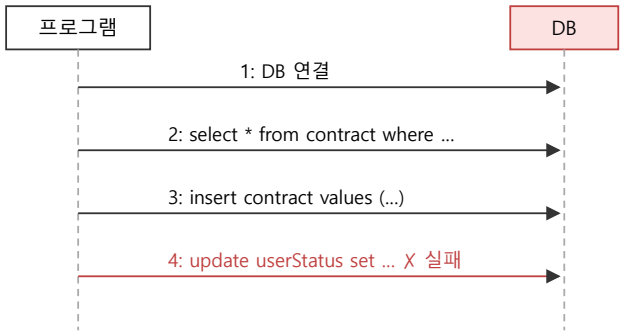


그림 7 (책 그림 3.13). 트랜잭션 없이 실행하다 4에서 실패하면 contract와 userStatus가 어긋난다

2에서 계약 존재 여부를 확인하고, 없으면 3(계약 추가)과 4(상태 변경)를 실행한다. 4가 실패하면 contract에는 데이터가 있지만 userStatus엔 반영되지 않는다. 사용자는 상태가 안 바뀌었으니 다시 계약을 시도하지만, 이미 contract에 데이터가 있어 **더 이상 진행할 수 없다**. 트랜잭션을 제대로 설정했다면 생기지 않았을 문제다.

일부 실패에도 커밋해야 하는 경우

보통 일부가 실패하면 전체를 롤백하지만, 일부 오류가 나도 커밋해야 할 때가 있다. 예) 회원 가입 후 환영 메일 발송 (sendMail은 문제 시 RuntimeException 발생).

```
@Transactional // 트랜잭션 범위
public void join(JoinRequest join) {
    ...
    memberDao.insert(member);   // DB에 데이터 추가
    mailClient.sendMail(...);   // 메일 발송
}
```

스프링 `@Transactional`은 런타임 예외가 나면 전체를 롤백한다 → 메일 서버의 일시적 문제만으로 **회원 가입 자체가 실패**한다. 원하는 동작이 아니라면 메일 오류를 따로 처리해 무시한다.

```
@Transactional
public void join(JoinRequest join) {
    ...
    memberDao.insert(member);
    try {
        mailClient.sendMail(...);
    } catch (Exception ex) {
        // 메일 발송 오류 무시, 로그로 기록해 모니터링
    }
}
```

다음 장 예고

외부 API 연동과 DB 작업이 섞이면(외부 호출은 성공했는데 DB 작업이 실패하는 등) 트랜잭션 처리가 복잡해진다 → 4장에서 다룬다.

6. 복습 퀴즈

1. 풀 스캔이 발생하는 대표적인 두 경우는?
   → where 조건에 맞는 인덱스가 없을 때 / DB가 전체 탐색이 더 빠르다고 판단할 때
2. 10년간 520건, 최대 3.34 TPS인 사내 공지 게시판에 category 인덱스가 필요한가?
   → 불필요. 데이터·동시 접속이 적어 풀 스캔도 문제없음
3. 사용자당 활동 로그가 수만 건 쌓인다면 단일·복합 중 무엇을?
   → (userId, activityDate) 복합 인덱스
4. 선택도란? 높을수록 좋은 이유는?
   → 칼럼의 고유 값 비율. 고유 값이 많을수록 인덱스로 걸러지는 데이터가 적어 효율적
5. select * 와 select activityDate, activityType 중 커버링 인덱스 효과를 보는 쪽은?
   → 후자. 필요한 칼럼이 모두 인덱스에 있어 실제 데이터를 읽지 않음
6. 효과가 적은 인덱스를 추가하면 생기는 비용 두 가지는?
   → 추가·변경·삭제 시 인덱스 관리 시간, 메모리·디스크 사용량 증가
7. name 인덱스 없이 예약자 조회를 빠르게 할 방법은?
   → 요구사항을 "특정 일자 + 이름"으로 바꿔 기존 reserveDate 인덱스 활용
8. 설문 30개 목록을 서브 쿼리로 조회하면 실행되는 쿼리 수와 시간은?
   → 61번, 0.01 + 0.1×30 + 0.05×30 = 4.51초
9. 선택도가 낮은데도 인덱스를 거는 예는?
   → 작업 큐의 status — 소수인 'W'만 반복 조회하므로
10. 커버링 인덱스의 정의와 장점은?
    → 쿼리에 필요한 칼럼을 모두 포함한 인덱스. 실제 데이터를 읽지 않아 빠름
11. answerCnt 같은 집계 칼럼을 두는 방식의 이름과, 증감 쿼리에서 확인할 점은?
    → 미리 집계(비정규화). DBMS·격리 수준에서 원자적으로 처리되는지 검증
12. `limit 10 offset 99990`이 느린 이유는?
    → 99,990개를 세고 나서야 10개를 읽기 때문
13. ID 기준 조회에서 "다음 데이터 있음"을 판단하는 방법은?
    → 1개 더(11개) 조회해 11개면 true
14. count(*)가 데이터 증가에 따라 느려지는 이유는?
    → 조건에 맞는 데이터를 전부 탐색해야 해서 (커버링이어도 인덱스 전체 스캔)
15. DELETE 후에도 디스크 용량이 줄지 않는 이유와 해결책은?
    → 삭제 표시만 하고 공간을 재사용. 최적화 작업으로 재배치
16. 주 DB-복제 DB 구조에서 쿼리를 어떻게 나누는가?
    → 변경은 주 DB, 조회는 복제 DB (단, 변경 기능 안의 조회는 주 DB)
17. 변경 기능에서 복제 DB로 SELECT하면 생기는 두 가지 문제는?
    → 복제 지연에 따른 불일치, 트랜잭션 커밋 전 데이터 불일치
18. 쿼리 타임아웃이 재시도 폭증을 막는 원리는?
    → 오래 걸리는 요청을 끊어 종료시키므로, 재시도해도 이전 요청이 쌓이지 않음
19. 결제 기능의 타임아웃을 짧게 잡으면 안 되는 이유는?
    → 결제 중 에러 시 후속 처리·정합성이 복잡해짐

20. 배치 집계 쿼리가 몇 시간째 안 끝날 때 쓸 수 있는 두 대안은?
→ 커버링 인덱스, 데이터를 일정 크기(시간 구간)로 나눠 처리

21. integer와 varchar 칼럼을 조인하면 어떤 문제가?
→ 행마다 타입 변환이 일어나 인덱스를 온전히 활용 못함

22. MySQL에서 대용량 테이블 변경이 위험한 이유는?
→ 새 테이블에 복사하는 동안 DML이 막혀 서비스가 멈춤 (복제 DB도 따로 변경 진행)

23. DB 최대 연결 100, 서버당 풀 30일 때 서버를 4대로 늘리면? 이때 연결 수를 늘리면 안 되는 조건은?
→ 20개 연결 실패 / DB CPU 70% 이상일 때

24. 환영 메일 실패로 회원 가입 전체가 롤백되지 않게 하려면?
→ 메일 발송을 try-catch로 감싸 오류를 무시하고 로그로 기록